

1) XV6 – LRU

i) proc.h

```
// Frame info structure for LRU
struct frameinfo {
    uint va;           // Virtual address of page
    pte_t *pte;        // Page table entry
    uint last_used;    // Time of last access
};

// In struct proc (add these lines near the bottom)
struct frameinfo frames[16]; // Track resident pages (up to 16)
int framecount;              // Number of frames used
```

ii) vm.c

```
extern uint ticks; // global tick counter from trap.c

void update_lru_access(struct proc *p, uint va) {
    for (int i = 0; i < p->framecount; i++) {
        if (p->frames[i].va == va) {
            p->frames[i].last_used = ticks;
            break;
        }
    }
}

int allocuvm(pte_t *pgdir, uint oldsz, uint newsz) {
    char *mem;
    uint a;

    if (newsz >= KERNBASE) return 0;
    if (newsz < oldsz) return oldsz;

    a = PGROUNDUP(oldsz);
    for (; a < newsz; a += PGSIZE) {
        mem = kalloc();
        if (mem == 0) {
            cprintf("allocuvm out of memory\n");
            deallocuvm(pgdir, newsz, oldsz);
            return 0;
        }
        memset(mem, 0, PGSIZE);
        if (mappages(pgdir, (char *)a, PGSIZE, V2P(mem), PTE_W | PTE_U) < 0) {
            cprintf("allocuvm out of memory (2)\n");
            deallocuvm(pgdir, newsz, oldsz);
            kfree(mem);
            return 0;
        }
    }
}
```

```

// ----- LRU Update -----
struct proc *curproc = myproc();
if (curproc) {
    if (curproc->framecount < 16) {
        curproc->frames[curproc->framecount].va = a;
        curproc->frames[curproc->framecount].pte = walkpgdir(pgdir, (void *)a, 0);
        curproc->frames[curproc->framecount].last_used = ticks;
        curproc->framecount++;
    } else {
        // Choose victim (least recently used)
        int victim_index = 0;
        for (int i = 1; i < curproc->framecount; i++) {
            if (curproc->frames[i].last_used < curproc->frames[victim_index].last_used) {
                victim_index = i;
            }
        }

        // Free victim
        pte_t *vppte = curproc->frames[victim_index].pte;
        uint vpa = PTE_ADDR(*vppte);
        kfree(P2V(vpa));
        *vppte = 0;

        // Replace with new
        curproc->frames[victim_index].va = a;
        curproc->frames[victim_index].pte = walkpgdir(pgdir, (void *)a, 0);
        curproc->frames[victim_index].last_used = ticks;
    }
}
return newsz;
}

```

iii) mytest.c

```

#include "types.h"
#include "user.h"

#define MAX_PAGES 5
#define TOTAL_ACCESSES 15

int main(int argc, char *argv[]) {
    int lru[MAX_PAGES];
    int last_used[MAX_PAGES];
    int i, j, page, hit = 0, miss = 0, time = 0;

    for (i = 0; i < MAX_PAGES; i++) {
        lru[i] = -1;
        last_used[i] = -1;
    }

    printf(1, "Starting LRU page replacement simulation...\n");
}

```

```

int accesses[TOTAL_ACCESSES] = {0,1,2,3,4,1,5,0,6,1,2,7,3,8,4};

for (i = 0; i < TOTAL_ACCESSES; i++) {
    page = accesses[i];
    time++;

    int found = 0;
    for (j = 0; j < MAX_PAGES; j++) {
        if (lru[j] == page) {
            found = 1;
            last_used[j] = time;
            break;
        }
    }

    if (found) {
        hit++;
        printf(1, "Access page %d: HIT\n", page);
    } else {
        miss++;
        int replaced = -1;
        for (j = 0; j < MAX_PAGES; j++) {
            if (lru[j] == -1) {
                lru[j] = page;
                last_used[j] = time;
                replaced = j;
                break;
            }
        }
        if (replaced == -1) {
            int lru_index = 0, min_time = last_used[0];
            for (j = 1; j < MAX_PAGES; j++) {
                if (last_used[j] < min_time) {
                    min_time = last_used[j];
                    lru_index = j;
                }
            }
            printf(1, "Access page %d: MISS, replacing page %d\n", page, lru[lru_index]);
            lru[lru_index] = page;
            last_used[lru_index] = time;
        } else {
            printf(1, "Access page %d: MISS, placed in free frame\n", page);
        }
    }
}

printf(1, "LRU simulation completed.\n");
printf(1, "Total hits: %d\n", hit);
printf(1, "Total misses: %d\n", miss);

exit();

```

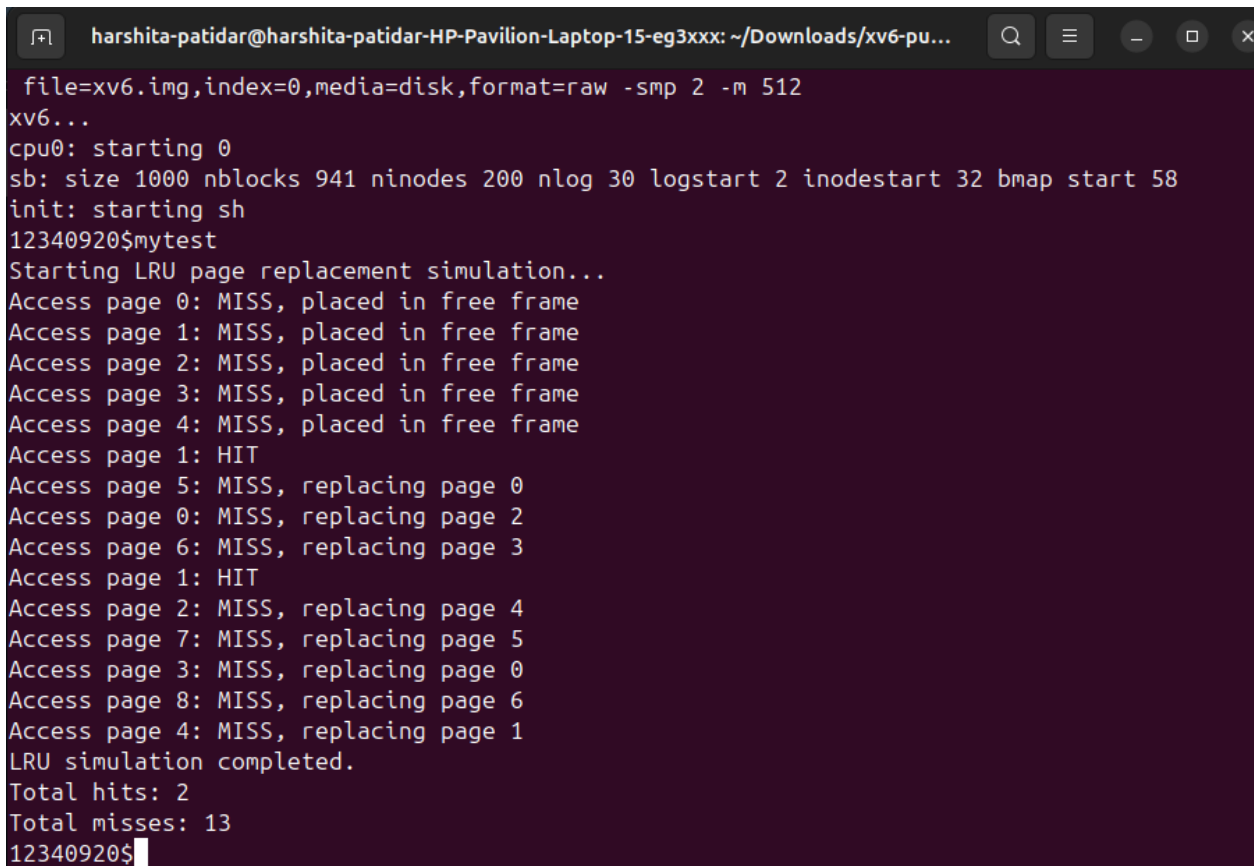
```
}
```

iv) Makefile

```
UPROGS=\
```

```
...
```

```
_mytest\
```

A terminal window with a dark purple background. The title bar shows the user 'harshita-patidar' on a 'harshita-patidar-HP-Pavilion-Laptop-15-eg3xxx' machine, with the current directory being '~/Downloads/xv6-pu...'. The terminal output shows the xv6 boot process: 'file=xv6.img,index=0,media=disk,format=raw -smp 2 -m 512', 'xv6...', 'cpu0: starting 0', 'sb: size 1000 nblocks 941 ninodes 200 nlog 30 logstart 2 inodestart 32 bmap start 58', 'init: starting sh', and a shell prompt '12340920\$'. The user enters 'mytest', which runs an LRU page replacement simulation. The simulation logs 13 misses and 2 hits for a sequence of page accesses (0, 1, 2, 3, 4, 1, 5, 0, 6, 1, 2, 7, 3, 8, 4). The simulation ends with 'LRU simulation completed.', 'Total hits: 2', and 'Total misses: 13'. The prompt returns to '12340920\$'.

Now XV6-FIFO

I) proc.h

```
#define MAX_PAGES 20 // Maximum resident pages per process
```

```
struct proc {  
    // existing members...  
  
    int pages[MAX_PAGES]; // FIFO pages (virtual addresses)  
    int page_count;        // Current number of pages loaded  
};
```

ii) vm.c

```
int allocvm(pde_t *pgdir, uint oldsz, uint newsz) {  
    char *mem;
```

```

uint a;

if(newsz >= KERNBASE) return 0;
if(newsz < oldsz) return oldsz;

a = PGROUNDUP(oldsz);
for(; a < newsz; a += PGSIZE) {
    mem = kalloc();
    if(mem == 0) {
        cprintf("allocvm out of memory\n");
        deallocvm(pgdir, newsz, oldsz);
        return 0;
    }
    memset(mem, 0, PGSIZE);

    if(mappages(pgdir, (char*)a, PGSIZE, V2P(mem), PTE_W|PTE_U) < 0) {
        kfree(mem);
        deallocvm(pgdir, newsz, oldsz);
        return 0;
    }

    struct proc *cur = myproc();
    if(cur) {
        if(cur->page_count < MAX_PAGES) {
            cur->pages[cur->page_count++] = (int)a;
            cprintf("Allocated page %d at virtual address 0x%x\n",
                cur->page_count-1, a);
        } else {
            // FIFO eviction → evict the first page
            int evict = cur->pages[0];

            // Free victim's memory
            pte_t *pte = walkpgdir(pgdir, (void*)evict, 0);
            if(pte && (*pte & PTE_P)) {
                char *v = (char*)P2V(PTE_ADDR(*pte));
                kfree(v);
                *pte = 0;
            }

            // Shift array (queue behavior)
            for(int i = 1; i < MAX_PAGES; i++)
                cur->pages[i-1] = cur->pages[i];

            cur->pages[MAX_PAGES-1] = (int)a;

            cprintf("Evicted page at 0x%x, allocated new page at 0x%x\n", evict, a);
        }
    }
}
return newsz;
}

```

iii) mytest.c

```
#include "types.h"
#include "user.h"

#define MAX_PAGES 3    // Number of frames
#define TOTAL_ACCESSES 12 // Total virtual accesses

int main(int argc, char *argv[]) {
    int fifo[MAX_PAGES];
    int next_to_replace = 0; // FIFO pointer
    int i, j, page;
    int hit = 0, miss = 0;

    // Init frames
    for(i = 0; i < MAX_PAGES; i++)
        fifo[i] = -1;

    printf(1, "Starting FIFO page replacement simulation...\n");

    int accesses[TOTAL_ACCESSES] = {1, 2, 3, 4, 1, 2, 5, 1, 2, 3, 4, 5};

    for(i = 0; i < TOTAL_ACCESSES; i++) {
        page = accesses[i];
        int found = 0;

        // Check for hit
        for(j = 0; j < MAX_PAGES; j++) {
            if(fifo[j] == page) {
                found = 1;
                break;
            }
        }

        if(found) {
            hit++;
            printf(1, "Access page %d: HIT\n", page);
        } else {
            miss++;
            printf(1, "Access page %d: MISS, replacing page %d\n",
                page, fifo[next_to_replace]);

            fifo[next_to_replace] = page;
            next_to_replace = (next_to_replace + 1) % MAX_PAGES;
        }
    }

    printf(1, "FIFO simulation completed.\n");
    printf(1, "Total hits: %d\n", hit);
    printf(1, "Total misses: %d\n", miss);
}
```

```
    exit();  
}
```

iv) Makefile

```
UPROGS=\
```

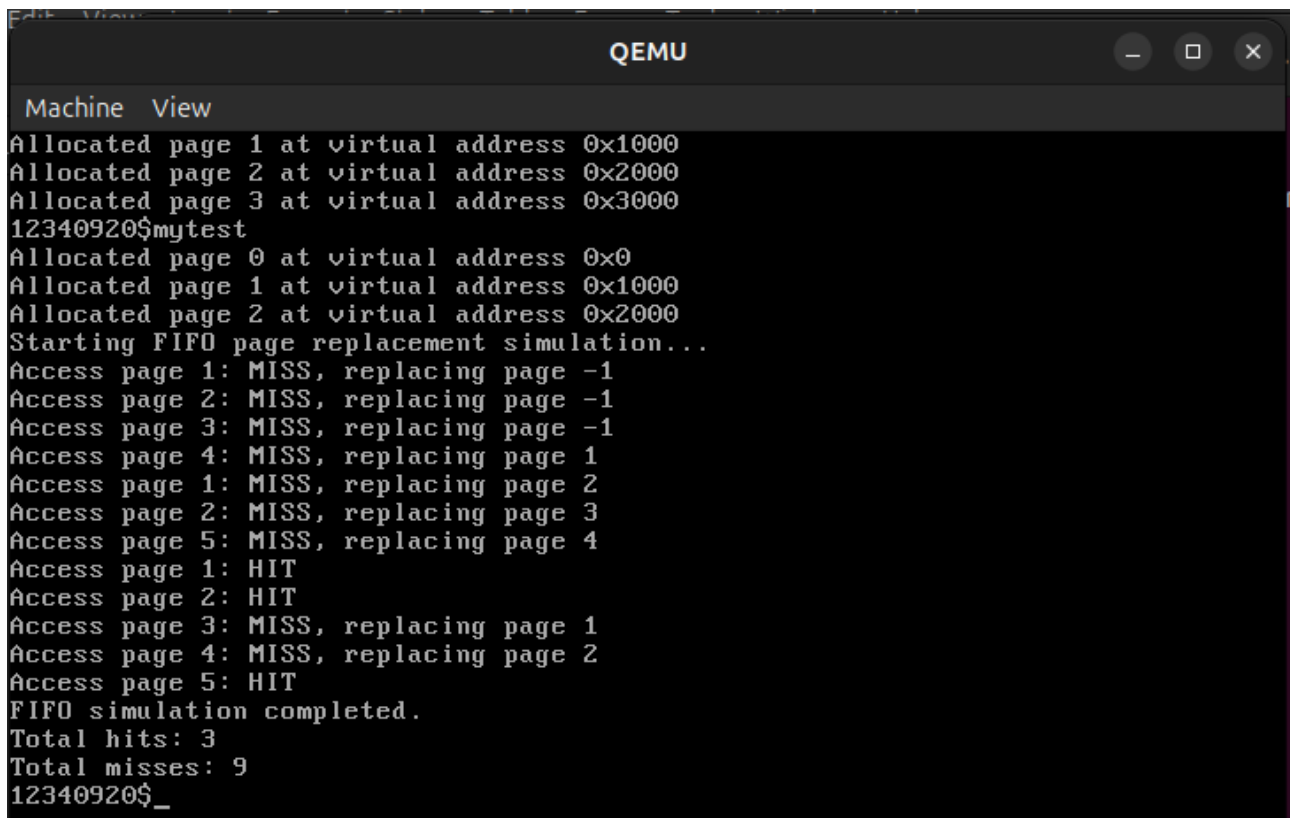
```
...
```

```
_mytest\
```

Observation about the Belady's anomaly.

Normally, we expect that increasing the cache size should improve the hit rate, since more pages can be stored. However, in the case of FIFO page replacement, an unusual situation occurs: when the cache size is increased from 3 to 4, the hit rate actually decreases instead of improving. This counterintuitive behavior is known as **Belady's anomaly**.

FOR MAX_PAGE=3



```
QEMU  
Machine View  
Allocated page 1 at virtual address 0x1000  
Allocated page 2 at virtual address 0x2000  
Allocated page 3 at virtual address 0x3000  
12340920$mytest  
Allocated page 0 at virtual address 0x0  
Allocated page 1 at virtual address 0x1000  
Allocated page 2 at virtual address 0x2000  
Starting FIFO page replacement simulation...  
Access page 1: MISS, replacing page -1  
Access page 2: MISS, replacing page -1  
Access page 3: MISS, replacing page -1  
Access page 4: MISS, replacing page 1  
Access page 1: MISS, replacing page 2  
Access page 2: MISS, replacing page 3  
Access page 5: MISS, replacing page 4  
Access page 1: HIT  
Access page 2: HIT  
Access page 3: MISS, replacing page 1  
Access page 4: MISS, replacing page 2  
Access page 5: HIT  
FIFO simulation completed.  
Total hits: 3  
Total misses: 9  
12340920$_
```

For MAX_PAGE=4

```
QEMU
Machine View

Allocated page 1 at virtual address 0x1000
Allocated page 2 at virtual address 0x2000
Allocated page 3 at virtual address 0x3000
12340920$mytest
Allocated page 0 at virtual address 0x0
Allocated page 1 at virtual address 0x1000
Allocated page 2 at virtual address 0x2000
Starting FIFO page replacement simulation...
Access page 1: MISS, replacing page -1
Access page 2: MISS, replacing page -1
Access page 3: MISS, replacing page -1
Access page 4: MISS, replacing page -1
Access page 1: HIT
Access page 2: HIT
Access page 5: MISS, replacing page 1
Access page 1: MISS, replacing page 2
Access page 2: MISS, replacing page 3
Access page 3: MISS, replacing page 4
Access page 4: MISS, replacing page 5
Access page 5: MISS, replacing page 1
FIFO simulation completed.
Total hits: 2
Total misses: 10
12340920$_
```

For MAX_PAGE=5


```
Allocated page 1 at virtual address 0x1000
Allocated page 2 at virtual address 0x2000
Allocated page 3 at virtual address 0x3000
12340920$mytest
Allocated page 0 at virtual address 0x0
Allocated page 1 at virtual address 0x1000
Allocated page 2 at virtual address 0x2000
Starting FIFO page replacement simulation...
Access page 1: MISS, replacing page -1
Access page 2: MISS, replacing page -1
Access page 3: MISS, replacing page -1
Access page 4: MISS, replacing page -1
Access page 1: HIT
Access page 2: HIT
Access page 5: MISS, replacing page -1
Access page 1: HIT
Access page 2: HIT
Access page 3: HIT
Access page 4: HIT
Access page 5: HIT
FIFO simulation completed.
Total hits: 7
Total misses: 5
12340920$_
```